Bài tập buổi 49 — Làm cho trang Profile sống dậy

> **Dự án:** CybersoftMarketPlace · .NET 10 · kiến trúc 4 tầng
> **Làm cả:** Backend (API) và Frontend (Blazor)
> **Thời gian:** 1 tuần

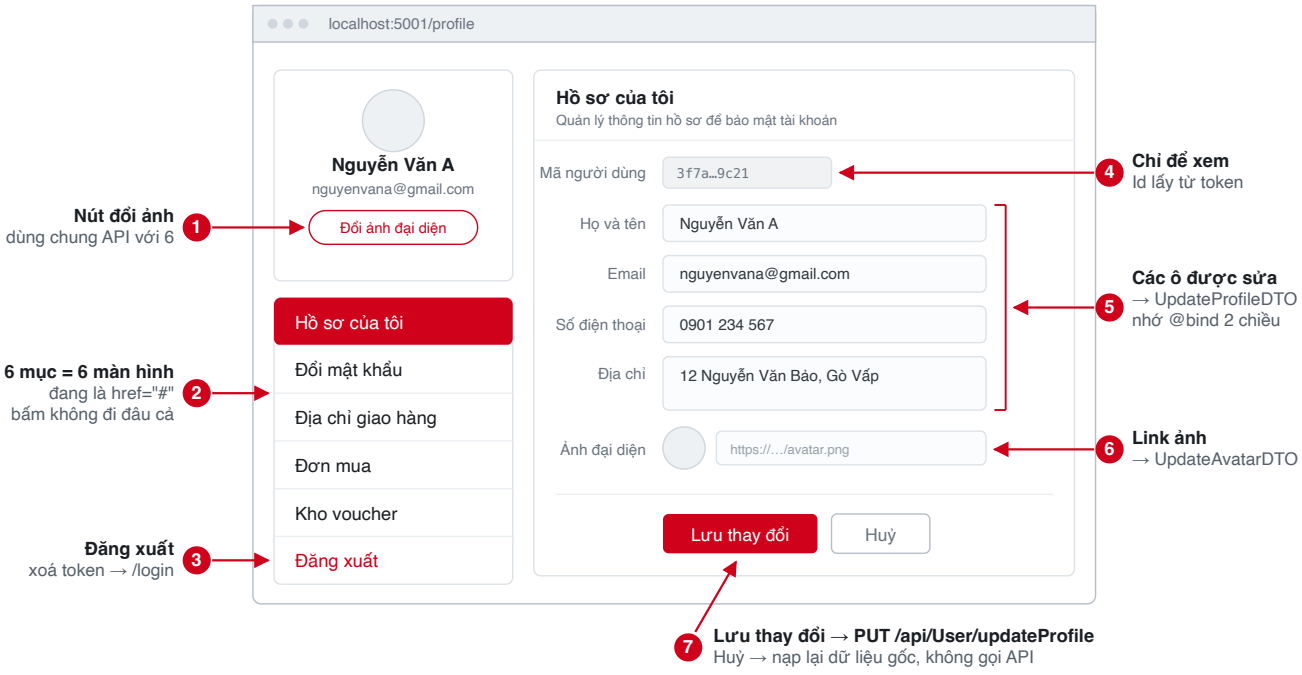
Buổi 47 lớp mình đã làm xong đăng ký — đăng nhập — sinh JWT, buổi 48 làm trang chi tiết sản phẩm. Trang `/profile` thì mình đã dựng sẵn giao diện bằng Bootstrap cho các bạn rồi, nhưng nói thật thì nó mới chỉ là cái vỏ: bấm vào đâu cũng không có gì xảy ra, gõ vào ô input xong bấm *Lưu thay đổi* thì… cũng không lưu gì cả.

Việc của các bạn tuần này là biến từng nút bấm trong cái vỏ đó thành chức năng chạy thật — từ giao diện, xuống API, xuống tận database rồi quay ngược lên.

Mở file `Profile.razor` ra, ngó qua một lượt, rồi nhìn cái hình bên dưới.

---

1. Giao diện hiện có — và những chỗ cần làm



Đi từng mũi tên một nhé.

1 — Nút *Đổi ảnh đại diện*

Hiện tại bấm vào không có gì. Bài này chưa cần upload file thật, cứ cho người dùng dán một cái link ảnh vào ô ở mũi tên **6** là được, nút này chỉ cần focus vào ô đó hoặc mở một modal nhỏ. Ai muốn làm upload `IFormFile` thật thì để dành làm phần nâng cao.

2 — Menu 6 mục bên trái

Đây là chỗ đáng làm nhất. Mở `Profile.razor:26-43` sẽ thấy cả 6 mục đều là `<a href="#">` — tức là trang trí thôi. Mỗi mục là một màn hình:

Mục	Phải làm gì
Hồ sơ của tôi	màn hình đang có sẵn
Đổi mật khẩu	tự dựng thêm, xem hình ở mục 2
Địa chỉ giao hàng	phần nâng cao
Đơn mua	tự dựng thêm, xem hình ở mục 2
Kho voucher	phần nâng cao
Đăng xuất	mũi tên 3

Chọn 1 trong 2 cách làm: hoặc dùng một biến `string tabDangChon` với `@onclick` rồi `@if` để ẩn hiện từng khối (dễ, làm nhanh); hoặc tách hẳn route riêng `/profile`, `/profile/password`, `/profile/orders`… mỗi màn một file `.razor` dùng chung layout (đúng chuẩn hơn, mình khuyến khích cách này).

3 — Đăng xuất

Cái này nhẹ nhất, không cần API. `UserStateService` đã có sẵn `LogoutAsync()` rồi — gọi nó để xoá token trong localStorage, rồi `NavigateTo("/login")`. Nhớ kiểm tra lại: sau khi đăng xuất mà gõ tay `/profile` vào thanh địa chỉ thì phải bị đá ra, không được để trang trắng hay văng lỗi null.

4 — Ô *Mã người dùng*

Ô này **chỉ để xem**, không cho sửa, và cũng đừng gửi nó lên server. `Id` của người đang đăng nhập nằm sẵn trong token rồi, server tự đọc ra bằng `_jwtAuthService.DecodePayloadTokenId(token)` — y hệt cách `UserService.cs:37` đang làm với `GetProfileAsync`.

Nghĩ thử xem: nếu API nhận `Id` do client gửi lên thì chuyện gì xảy ra khi ai đó sửa `Id` thành của người khác rồi bấm Lưu?

5 — Bốn ô *Họ và tên* / *Email* / *SĐT* / *Địa chỉ*

Đây là phần chính. Hai việc phải làm:

Ở FE, các ô này đang dùng `value="@_userStateService.CurrentUser.FullName"` — một chiều, gõ vào không lưu lại được gì. Phải đổi sang `@bind`. Và nghĩ tiếp: bind thẳng vào `CurrentUser` hay copy ra một object `UpdateProfileDTO` riêng rồi bind vào đó? Cứ thử cả hai rồi bấm nút *Huỷ* xem cách nào khôi phục dữ liệu dễ hơn.

Ở BE, viết `PUT /api/User/updateProfile`. DTO gợi ý:

```
public class UpdateProfileDTO
{
    public string FullName { get; set; } = null!;
    public string? Phone { get; set; }
    public string? Address { get; set; }
    // Còn Email thì sao? Cho sửa hay không cho sửa?
    // Nếu cho sửa thì phải check trùng như lúc Register — nhớ luôn cả check trùng Phone.
    // Còn Id thì tuyệt đối không đưa vào đây, lý do đã nói ở mũi tên 4.
}
```

6 — Ô link ảnh đại diện

Tách riêng một API `PUT /api/User/updateAvatar` với `UpdateAvatarDTO` chỉ đúng một field `AvatarUrl`. Có bạn sẽ hỏi: sao không gộp luôn vào `UpdateProfileDTO` cho gọn? Gộp cũng được, nhưng khi lên phần nâng cao làm upload file thật thì avatar đi đường riêng (multipart) chứ không đi chung JSON với các field kia nữa — tách sẵn từ giờ đỡ phải sửa lại.

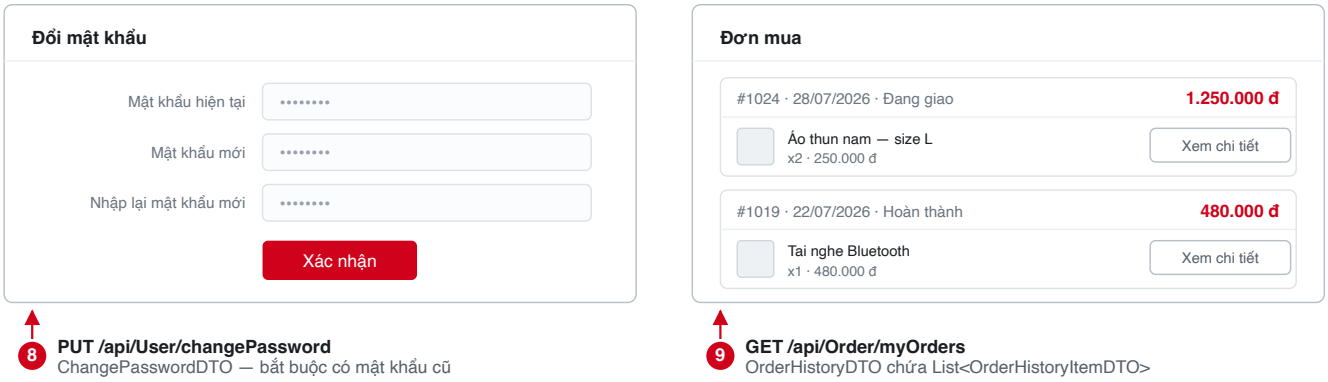
7 — Hai nút cuối trang

Lưu thay đổi gọi API, còn *Huỷ* thì không gọi API nào cả, chỉ nạp lại dữ liệu cũ vào form. Trong lúc đang gọi API nhớ cho nút `disabled` và hiện spinner — cách làm loading đã có sẵn ở `ProductDetail.razor` buổi trước. Gọi xong: thành công thì alert xanh, thất bại thì alert đỏ và hiện đúng `Message` mà API trả về, đừng nuốt lỗi.

---

2. Hai màn hình phải tự dựng thêm

Hai màn này chưa có gì cả, các bạn tự vẽ bằng Bootstrap. Đây là hình mình phác cho dễ hình dung, không cần giống y hệt:



8 — Đổi mật khẩu

Ba ô, và cái ô *Mật khẩu hiện tại* là bắt buộc phải có — đừng bỏ đi cho tiện. Cứ thử nghĩ: nếu bỏ đi thì một người mượn được máy bạn lúc bạn quên đăng xuất sẽ làm được gì?

```
public class ChangePasswordDTO
{
    public string OldPassword { get; set; } = null!;
    public string NewPassword { get; set; } = null!;
    public string ConfirmPassword { get; set; } = null!;
}
```

Bên service dùng `HelperFunction.VerifyPassword(old, user.PasswordHash)` để đối chiếu mật khẩu cũ, rồi `HelperFunction.HashPassword(new)` để lưu cái mới — hai hàm này đã viết sẵn trong `HelperFunc.cs`, cứ dùng lại. Còn `ConfirmPassword` thì kiểm ở FE hay BE? Câu trả lời là cả hai, và các bạn phải giải thích được vì sao.

9 — Đơn mua

Lấy danh sách đơn hàng của chính người đang đăng nhập (lại là `Id` lấy từ token, không nhận từ client). Bảng `Order` — `OrderItem` — `ProductVariant` — `Product` đã có sẵn dữ liệu, `OrderRepository` và `OrderItemRepository` cũng đã nằm sẵn trong UnitOfWork rồi, không phải tạo mới.

DTO thì làm theo kiểu lồng nhau, y như `ProductDetailDTO` các bạn đã viết ở buổi 48:

```
public class OrderHistoryDTO
{
    public int OrderId { get; set; }
    public DateTime CreatedAt { get; set; }
    public decimal TotalAmount { get; set; }
    public string Status { get; set; } = "Đang xử lý";  // bảng Order chưa có cột này, tự xử lý
    public List<OrderHistoryItemDTO> Items { get; set; } = new();
}

public class OrderHistoryItemDTO
{
    public int VariantId { get; set; }
    public string ProductName { get; set; } = null!;   // đi từ Variant.Product.Name
    public string VariantName { get; set; } = null!;
    public string ImageUrl { get; set; } = null!;
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
    // SubTotal = Quantity * UnitPrice thì để BE tính sẵn hay để FE tự nhân? Chọn một và giải
        thích.
}
```

Một cái bẫy ở đây: nếu các bạn lặp qua `order.OrderItems` rồi lại chấm tiếp `item.Variant.Product.Name`, EF Core sẽ bắn ra một đống câu SQL con — đúng bài N+1 mà lớp đã nói. Dùng `Include()` / `ThenInclude()`, và vì đây là màn hình chỉ đọc nên nhớ thêm `AsNoTracking()`.

---

3. Chuyện thiết kế DTO

Phần này mình muốn các bạn nghĩ kỹ nhất, vì code thì ai cũng viết được, còn thiết kế sai DTO thì sửa rất mệt về sau.

Nhìn giao diện để suy ra DTO, đừng nhìn bảng database. Bảng `User` có 12 cột, nhưng màn hình *Hồ sơ của tôi* chỉ cho sửa 3–4 cái. DTO nhận vào chỉ nên có đúng 3–4 field đó thôi. Một field có mặt trong DTO đầu vào đồng nghĩa với "field này cho phép sửa" — nên đưa dư một field là mở toang một cánh cửa.

DTO đi vào và DTO đi ra là hai class khác nhau. `ProfileUserDTO` (server trả về) và `UpdateProfileDTO` (client gửi lên) trông na ná nhau, nhưng đừng gộp làm một. Chúng sẽ tiến hoá theo hai hướng khác nhau ngay khi thêm chức năng mới.

Đừng bao giờ trả entity ra ngoài API. Trả thẳng `User` là kèm luôn `PasswordHash` cho cả thế giới xem. Trong dự án này quy định rõ: entity không rời khỏi tầng Application, controller chỉ nhận DTO và trả DTO, tất cả bọc trong `HTTPResponseData<T>`.

Điền thử cái bảng này trước khi bắt tay vào code:

Field trong `User`	Có trong `ProfileUserDTO` (trả ra)?	Có trong `UpdateProfileDTO` (nhận vào)?	Vì sao
`Id`	có	?	
`Username`	?	?	
`Email`	có	?	
`PasswordHash`	**không**	**không**	lộ hash là lộ mật khẩu
`Phone`	có	?	
`Address`	có	?	
`Avatar`	có	?	
`CreatedAt`	?	?	
`Deleted`	?	?	

---

4. Tổng hợp những API phải viết

#	Endpoint	DTO vào	DTO ra	Mức
1	`PUT /api/User/updateProfile`	`UpdateProfileDTO`	`ProfileUserDTO`	bắt buộc
2	`PUT /api/User/changePassword`	`ChangePasswordDTO`	`string`	bắt buộc
3	`PUT /api/User/updateAvatar`	`UpdateAvatarDTO`	`string`	bắt buộc
4	`GET /api/Order/myOrders`	—	`List<OrderHistoryDTO>`	khá

#	Endpoint	DTO vào	DTO ra	Mức
5	`GET /api/Order/{id}`	—	`OrderDetailDTO`	khá
6	API địa chỉ giao hàng (nhiều địa chỉ, 1 mặc định)	tự thiết kế	tự thiết kế	nâng cao
7	API kho voucher	tự thiết kế	tự thiết kế	nâng cao

Hai dòng nâng cao phải thêm bảng mới vào database — làm theo skill `/migration`, và **hỏi mình trước khi chạy** `dotnet ef database update`, đừng tự ý update.

---

5. Vài điều bắt buộc khi code

Backend. Đi đúng luồng `Controller → Service → UnitOfWork → Repository`, không được tắt đường. Controller chỉ gọi service rồi `return StatusCode(response.statusCode, response)` — thấy `if` nghiệp vụ nào trong controller là mình trừ điểm. Endpoint nào cũng phải có `[Authorize]` và `/// <summary>` cho Swagger. Trả đúng mã: 200 ok, 400 dữ liệu sai, 401 token hỏng, 404 không tìm thấy. Các câu thông báo thì thêm hằng số vào `UserResponseMessageDTO` trong `HTTPResponseData.cs`, đừng rải chuỗi khắp nơi.

Làm xong BE thì test hết trên Swagger trước đã (nhớ bấm *Authorize* dán token vào), chạy ngon rồi mới đụng tới FE. Đừng làm song song cả hai rồi lỗi không biết lỗi ở đâu.

Frontend. Logic để trong `UserStateService`, file `.razor` chỉ gọi service — đúng như cách lớp đang làm với `LoginAsync` và `GetProfileAsync`. Thêm vào đó mấy hàm kiểu:

```
public async Task<bool> UpdateProfileAsync(UpdateProfileDTO model)   { ... }
public async Task<bool> ChangePasswordAsync(ChangePasswordDTO model) { ... }
public async Task<bool> UpdateAvatarAsync(string avatarUrl)          { ... }
```

Gọi API xong nhớ `StateHasChanged()` để giao diện vẽ lại — cơ chế `OnChange` đã dựng sẵn trong `UserStateService.cs` rồi. Giao diện thì chỉ dùng HTML với Bootstrap 5 có sẵn, không thêm file CSS, không cài thư viện ngoài.

---

6. Chấm điểm

Nội dung	Điểm
DTO thiết kế đúng: tách vào/ra, không lộ `PasswordHash`, không nhận `Id` từ client	2.5
Đúng kiến trúc 4 tầng, controller sạch, không đụng DbContext ngoài repository	2.0
Mũi tên 1–7 chạy thật: bấm ở FE → dữ liệu đổi trong DB → F5 vẫn còn	2.5
FE: logic nằm ở StateService, `@bind` hai chiều, có loading và báo lỗi tử tế	1.5
Bảo mật: `[Authorize]`, BCrypt, kiểm mật khẩu cũ, validate đầu vào	1.0
Đơn mua (mũi tên 9) và xử lý được N+1	0.5
Phần nâng cao, code sạch, comment rõ ràng	+1.0

Trừ điểm: commit `bin/` `obj/` (–0.5) · trả entity ra API (–1.0) · lưu mật khẩu dạng plaintext (–2.0) · tự chạy `dotnet ef database update` khi chưa hỏi (–1.0).

---

7. Nộp bài

Tạo nhánh `buoi49_profile_<tên_của_bạn>`, commit message tiếng Việt mô tả rõ việc đã làm, kèm ảnh chụp màn hình Swagger đã test xong từng endpoint.

---

8. Mấy câu mình sẽ hỏi lúc chấm

1. Sao không dùng luôn entity `User` làm DTO cho nhanh?
2. `UpdateProfileDTO` của bạn có `Id` không? Nếu có, mình đổi `Id` đó thành của người khác rồi gửi lên thì chuyện gì xảy ra?
3. Đổi mật khẩu xong, token cũ còn xài được không? Theo bạn thì nên hay không nên, và làm thế nào?
4. `ProfileUserDTO` đang để `Id` kiểu `string` trong khi entity là `Guid` — vì sao vậy, có vấn đề gì không?
5. Màn *Đơn mua* của bạn chạy bao nhiêu câu SQL cho 10 đơn hàng? Mở log Serilog ra chứng minh.